

ADVANCED PROGRAMMING · FINAL CASE STUDY

Car Price Prediction Model

Training a regression model on ~99,000 real UK used-car listings to predict price from specifications

SRH Berlin University of Applied Sciences · M.Sc. Big Data & Artificial Intelligence · Group 5 · 28 July 2026

Presentation Roadmap

Five sections, four presenters — one continuous flow from problem to solution.

01 **Project Introduction**
The problem, motivation and objectives

02 **Dataset Overview & EDA**
Source, features and exploratory analysis

03 **Methodology & Implementation**
Preprocessing, feature selection, models

04 **Results & Discussion**
Model comparison and what drives price

05 **Conclusion & Future Work**
Key takeaways and next steps

UP NEXT · PART 01

Project Introduction

Why used-car pricing is a real problem — and what we set out to build.

01

Project Introduction

The Problem

Pricing a used car is genuinely hard. Its value depends on many interacting factors — age, mileage, engine size, fuel type and brand — and no single rule captures them all. Buyers risk overpaying and sellers risk underpricing.

A regression model can learn these patterns from thousands of past listings and estimate a fair price automatically.

£16,800

average price

£450–159,999

price range

This is a supervised regression task: the target we predict is a continuous number — the price in £.

Our Objectives

Build a price predictor
Train regression models to estimate price from specs

Compare several models
Benchmark a baseline against ensemble methods

Validate rigorously
Use 5-fold cross-validation for reliable scores

Explain the drivers
Identify which features most affect price

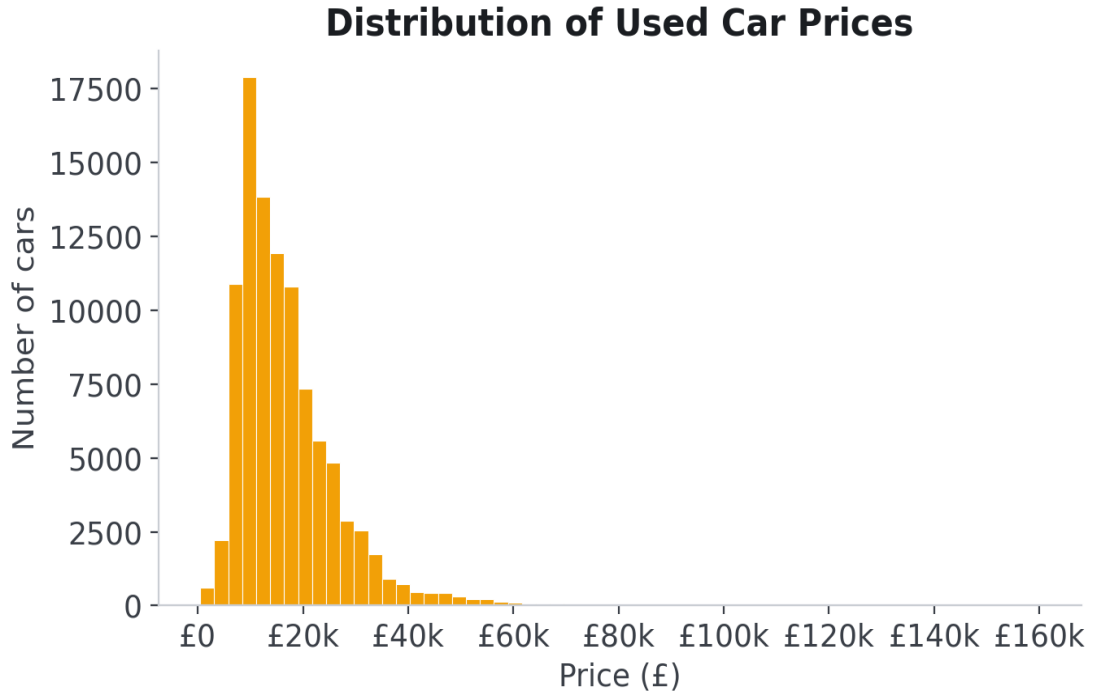
Dataset Overview

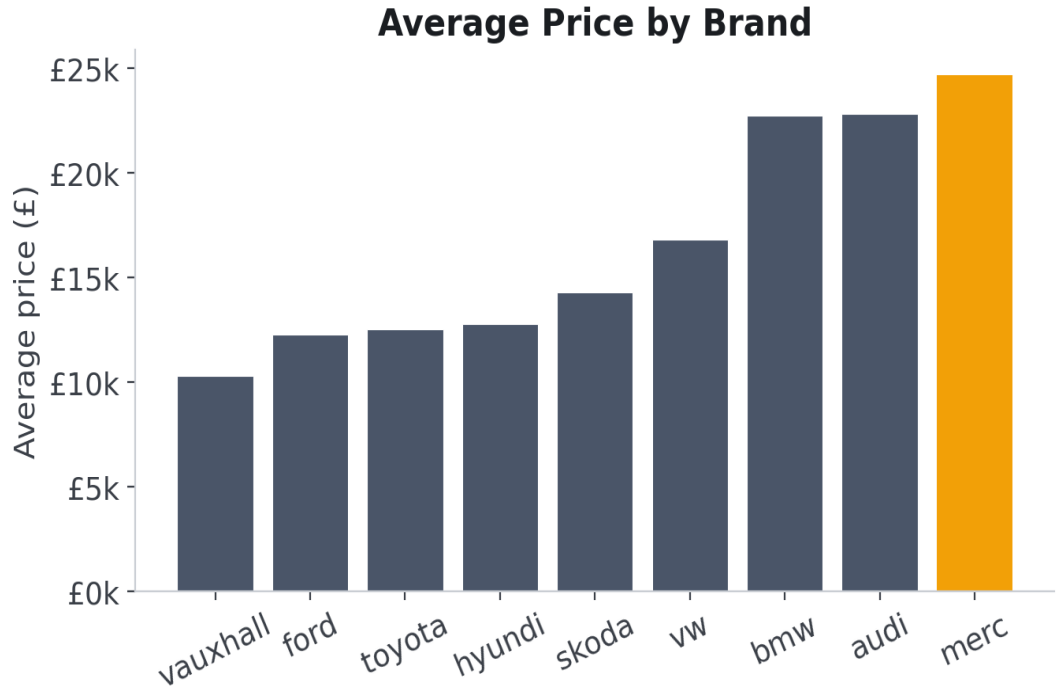
100,000 UK Used Car Data set — real listings scraped from the UK market, sourced from Kaggle.

97,442	9	8	£16,800
clean records	car brands	input features	average price

Feature	Type	Description
`price`	target	Sale price in £ — the value we predict
`year`	numeric	Registration year (age of the car)
`mileage`	numeric	Distance driven
`engineSize`	numeric	Engine size in litres
`mpg`	numeric	Fuel economy (miles per gallon)
`tax`	numeric	Annual road tax (£)
`brand`	category	One of 9 makes (Audi, BMW, Ford …)
`transmission`	category	Manual / Automatic / Semi-Auto
`fuelType`	category	Petrol / Diesel / Hybrid / Electric

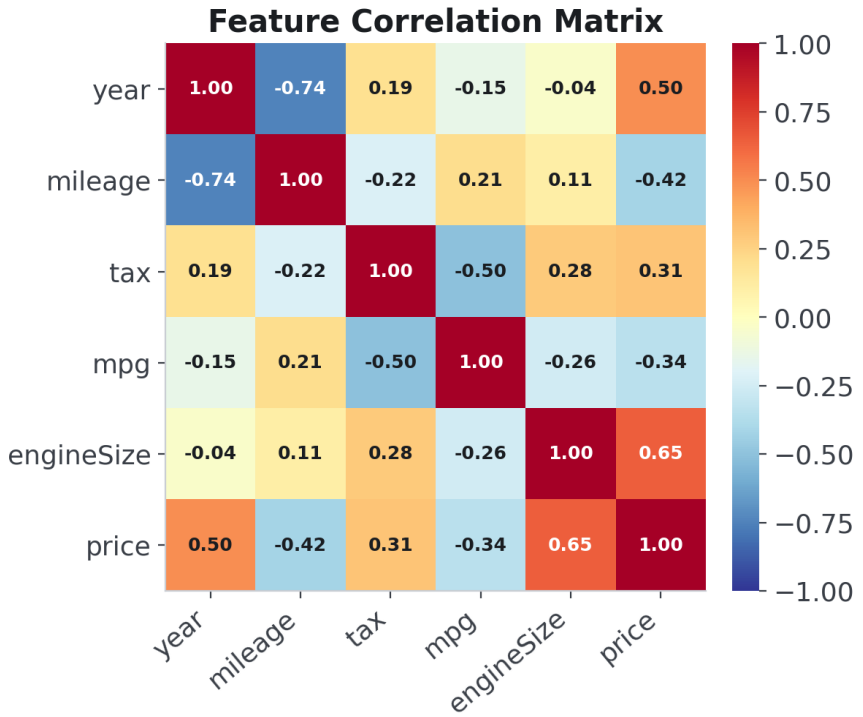
Exploratory Analysis — The Price Landscape

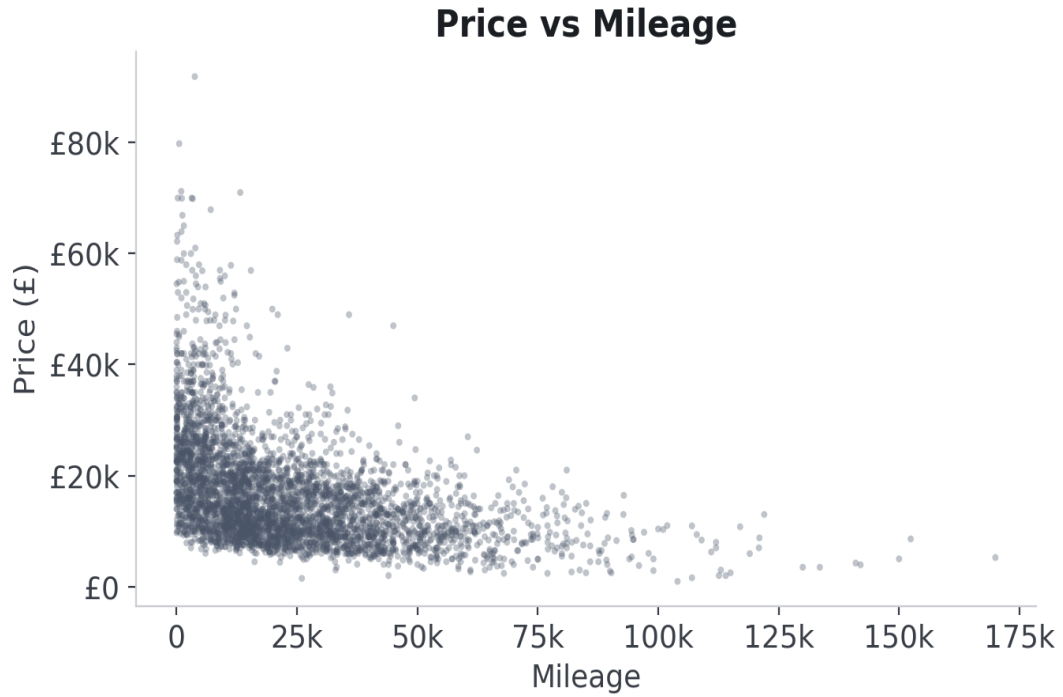




Prices are right-skewed — most cars sell for £5k–£25k, with a long tail of luxury models up to £160k. Average price varies widely by brand, with premium makes (Mercedes, BMW, Audi) commanding the highest prices.

How Features Relate to Price





Newer cars (higher year) and larger engines push price up; higher mileage pulls it down. The correlation matrix confirms these relationships, guiding which features the model should rely on.

UP NEXT · PART 02

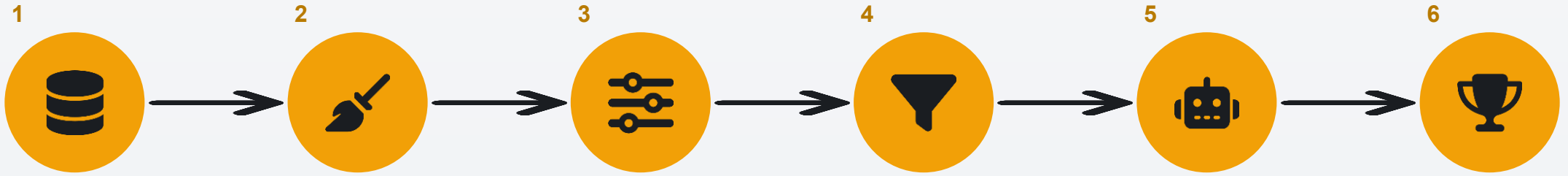
Methodology & Implementation

How we cleaned the data, engineered features, and trained our models.

02

Methodology — Our ML Pipeline

A clean, repeatable pipeline takes the raw CSV files all the way to an evaluated model.



Why a pipeline? Wrapping every step in one scikit-learn Pipeline means the exact same transformations are applied during training, cross-validation and testing — no data leakage, and the whole process is reproducible with a single call.

Data Preprocessing

Remove duplicates
1,475 duplicate listings dropped

Fix invalid values
Impossible years & zero-priced rows removed

Impute missing values
Median (numeric) & most-frequent (categorical)

Scale numeric features
StandardScaler normalises the ranges

Encode categoricals
One-Hot for brand, transmission, fuel type

preprocess.py

```
# numeric: fill gaps, then normalise
num = Pipeline([
  ('impute', SimpleImputer(
             strategy='median')),
  ('scale',  StandardScaler())])

# categorical: fill, then one-hot
cat = Pipeline([
  ('impute', SimpleImputer(
       strategy='most_frequent')),
  ('onehot', OneHotEncoder(
     handle_unknown='ignore'))])

# apply each to the right columns
prep = ColumnTransformer([
  ('num', num, num_features),
  ('cat', cat, cat_features)])
```

Feature Selection & Data Split

Features Used

NUMERIC

year · mileage · engineSize · mpg · tax

CATEGORICAL

brand · transmission · fuelType

Why we excluded the 'model' column

Deliberate design decision: 'model' has 194 unique values — one-hot encoding it would inflate the feature space dramatically. Excluding it keeps the baseline robust and reproducible.

Acknowledged trade-off: variants within a brand (e.g. Audi A1 vs A6) are not distinguished, reducing prediction granularity.

Addressed in Future Work via target encoding, which captures model-level signal without feature explosion.

Train / Test Split (80 / 20)

77,953

training cars (80%)

19,489

test (20%)

split.py

```
# 20% held out for honest testing
X_train, X_test, y_train, y_test = \
  train_test_split(
    X, y, test_size=0.2,
    random_state=42)
```

Models & 5-Fold Cross-Validation

Linear Regression

Simple baseline — assumes a straight-line relationship between features and price.

Random Forest

Ensemble of decision trees; captures non-linear patterns and interactions.

Gradient Boosting

Trees built sequentially, each correcting the last — strong on tabular data.

train.py

```
# 5-fold cross-validation loop
kf = KFold(n_splits=5, shuffle=True,
           random_state=42)
for name, model in models.items():
    pipe = Pipeline([('prep', prep),
                     ('model', model)])
    scores = cross_val_score(
        pipe, X_train, y_train,
        cv=kf, scoring='r2')
```

5-fold cross-validation

trains and tests each model on 5 different splits, so the score reflects real performance — not one lucky split.

Live Code Walkthrough

At this point we switch to the Python environment and run the full notebook live.

Load & merge
9 CSVs → 97,442 rows

EDA charts
generated live in the notebook

Preprocessing
pipeline runs before your eyes

Training & CV
3 models, real scores appear

`car_price_prediction.ipynb` — every cell is commented, runs top-to-bottom, and reproduces every number in this deck.

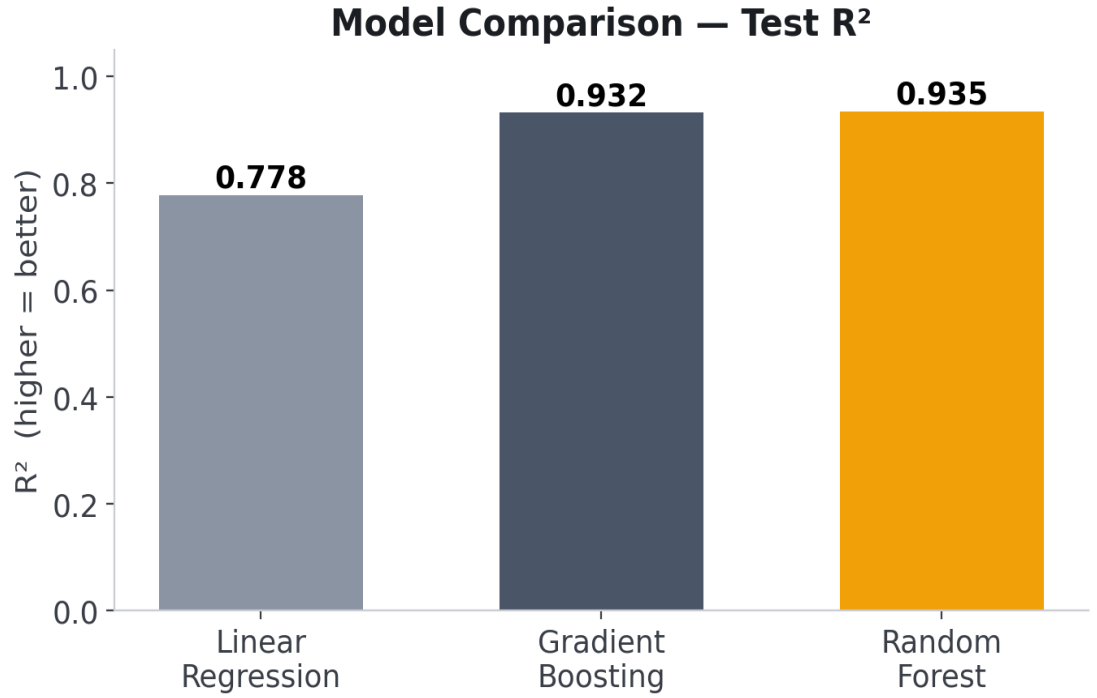
UP NEXT · PART 03

Results & Discussion

Which model won, how accurate it is, and what actually drives price.

03

Results — Model Comparison



Model	CV R^2	Test R^2	RMSE	MAE
Linear Regression	0.780	0.778	£4,647	£2,874
Gradient Boosting	0.928	0.932	£2,581	£1,644
Random Forest ★	0.941	**0.935**	£2,512	**£1,334**

Random Forest wins. Highest R^2 and lowest error — and cross-validation agrees with the test set.

The Model in Action — Real Test Cars

Three unseen cars from the test set — the model never saw their prices during training.

Ford Focus

2015 · 31,974 mi · Manual · Petrol 1.0L

ACTUAL	PREDICTED
£8,498	£8,350

VW T-Roc

2018 · 23,385 mi · Manual · Diesel 2.0L

ACTUAL	PREDICTED
£20,489	£19,590

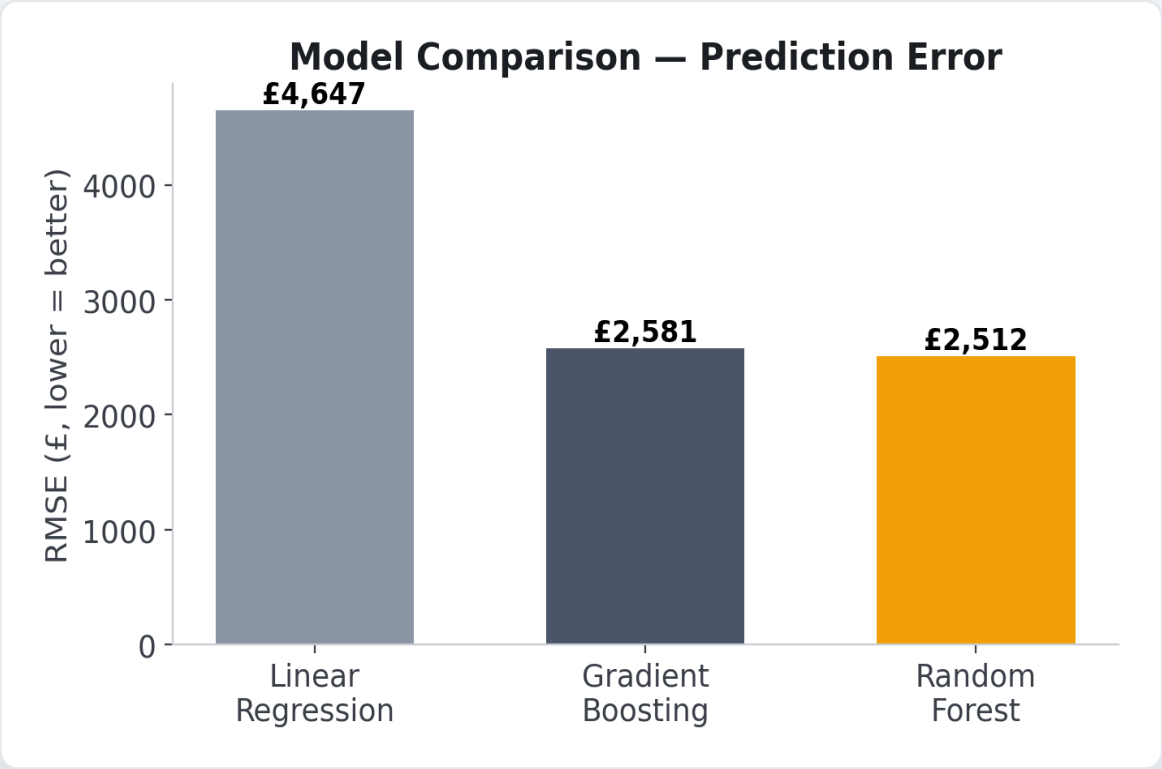
Mercedes SL Class

2019 · 1,000 mi · Semi-Auto · Petrol 2.0L

ACTUAL	PREDICTED
£32,299	£33,800

Across all 19,489 test cars the typical miss is £1,334 — these three are representative, not cherry-picked.

The Winning Model — Random Forest



0.935

Test R^2

of price variation explained

£1,334

Mean Abs. Error

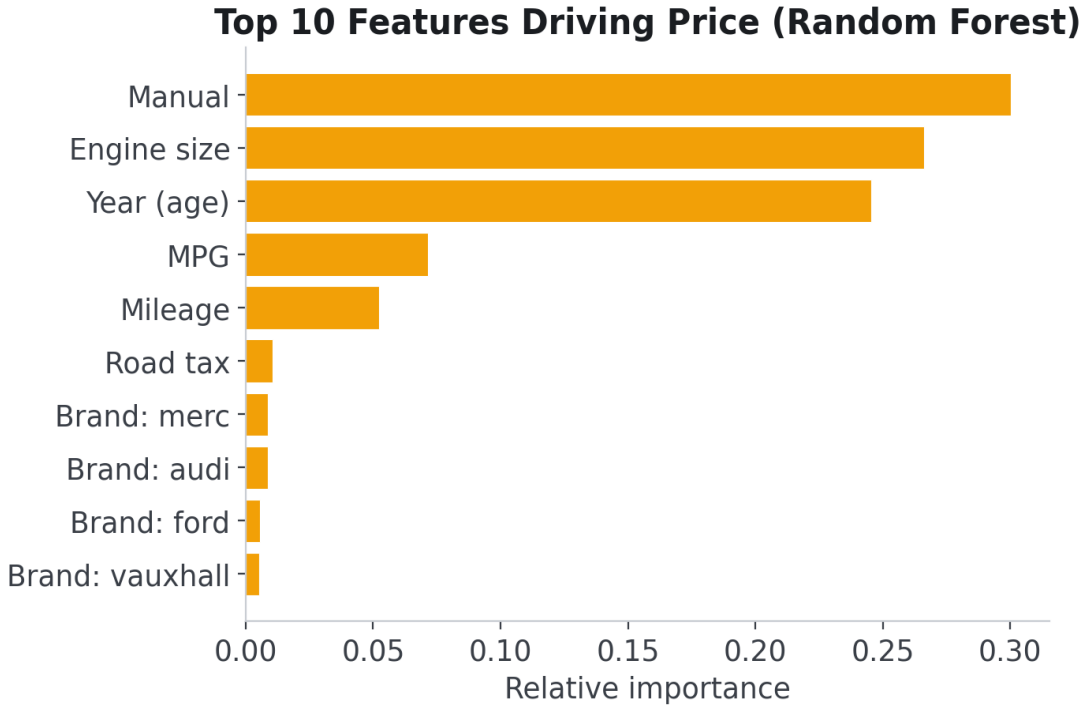
typical prediction is off by this

£2,512

RMSE

penalises the larger misses

What Actually Drives the Price?



Transmission & engine size lead

Together 56.9% of total importance (30.3% + 26.6%)

Vehicle age follows closely

Year: 24.5% — mpg (7.2%) and mileage (5.2%) add smaller contributions

Consistent with domain intuition

The ranking mirrors how the used-car market actually prices vehicles

Note: all nine brand indicators combined account for only 4.4%. Impurity-based importances measure predictive contribution within this model — not causal effects.

UP NEXT · PART 04

Conclusion & Future Work

What we achieved, what we learned, and where this could go next.

04

Conclusion — Key Takeaways

Real data, full pipeline

Merged and cleaned ~99k UK listings into 97,442 rows with a reproducible scikit-learn pipeline.

Random Forest performed best

Test R^2 = 0.935 with a mean absolute error (MAE) of £1,334 — a strong, reliable predictor of used-car price.

Validated, not lucky

5-fold cross-validation confirmed the results are stable and generalise to unseen cars.

Price is driven by the car itself

Transmission, engine size and age explain most of the price — matching real-world intuition.

Limitations & Reflection

An honest account of what our model cannot do — and why we accepted each trade-off.

One market, one period

The data covers nine UK brands, registered 1996–2020. The method transfers to any market, but the learned prices do not — a German model would need German listings.

Reduced granularity

Excluding the 194-value 'model' column means an Audi A1 and an A6 share the same brand signal, so within-brand differences are not fully captured.

Untuned by choice

We kept scikit-learn defaults to keep the baseline reproducible. Tuning would likely add accuracy — it is the first item in our future work, not an oversight.

Uneven coverage

Petrol and diesel dominate the data; electric and rare models are underrepresented, so predictions for them are less reliable.

Asking prices, not sale prices

Scraped listings show what sellers ask, not what buyers finally paid. Our model therefore predicts the market asking price.

Association, not causation

Feature importances show what the model relies on to predict — they do not prove that changing a feature would change a car's value.

Future Improvements

The model works well — here is how we would push it further.

Add the car 'model'

Use target encoding to include model without exploding the feature space.

Tune hyper-parameters

Grid/random search on tree depth and count to squeeze out more accuracy.

Try XGBoost / LightGBM

Modern gradient-boosting libraries often edge out Random Forest on tabular data.

Log-transform the price

Handle the right-skew so the model treats cheap and luxury cars more evenly.

Deploy as a web app

Wrap the model in a simple interface so anyone can get a price estimate.

Add more brands & years

Broaden the data to generalise beyond these 9 UK makes.

Sources & References

DATASET	**100,000 UK Used Car Data set — A. Desai, Kaggle (2020)**	kaggle.com/datasets/adityadesai13/used-car-dataset-ford-and-mercedes
LIBRARY	**scikit-learn — Pedregosa et al., machine-learning models & pipeline**	scikit-learn.org
LIBRARY	**pandas — data loading, merging and cleaning**	pandas.pydata.org
LIBRARY	**NumPy — numerical computing**	numpy.org
LIBRARY	**Matplotlib — data visualisation**	matplotlib.org
LANGUAGE	**Python 3 — implementation language**	python.org

All datasets and libraries are open-source and publicly available. Dataset used under its Kaggle licence for academic purposes.



Thank You

Questions & Discussion

Group 5

SRH Berlin University of Applied Sciences · M.Sc. Big Data & Artificial Intelligence · Advanced Programming · 2026

Questions

01 Why is a Random Forest better suited than a Linear Regression model for predicting used car prices?

02 How do the train/test split and 5-fold cross-validation work together to ensure our results are reliable and not simply due to chance?

03 Our feature importance analysis showed that the car's brand contributes only 4.4%. Why does the brand appear to matter so little when predicting a car's price?